PIIPS

Precision Intelligent Invoice Processing Suite · v2.2

Getting PIIPS running on a new UAT or Live server, and keeping an existing one in sync.

Overview

Three pieces move independently: the git branch (code), the staged build (Publish menu), and the actual server (IIS + the PIIPS_Backend service).

ENVIRONMENT	GIT BRANCH	TYPICAL SERVER PATH
UAT	Development	`E:\PIIPS_Application`
Live	uat	`E:\Source\piips`

Publish-to-UAT always deploys the latest local work (it commits/pushes any uncommitted edits to Development first). Publish-to-Live is a pure promotion of whatever the `uat` branch already has — it never touches Development, so something has to merge Development's work into `uat` before Live will ever see it.

Day-to-day: sync a staged build to the server

After Publish stages a build in `PIIPS_Published\<uat|live>`, mirror that folder onto the real server (not the repo root):

```
robocopy $StagedBuild $Destination /MIR /XJ /R:2 /W:5 /MT:8 `
  /XD "venv" ".venv" "__pycache__" "node_modules" "logs" `
  /XF "config.json" "config_nonencrypted.json" "*.log"

Restart-Service PIIPS_Backend
Start-Sleep -Seconds 15
Invoke-RestMethod http://127.0.0.1:8000/health   # expect {"status":"Healthy"}
```

`/MIR` (not a plain copy) deletes anything stale left over from a previous build — old content-hashed JS bundle filenames in particular. `config.json` is excluded from both the copy and the delete pass: the server keeps its own DPAPI-encrypted connection string.

> Then hard-refresh the browser (Ctrl+F5) or open a private window; if it's still stale, unregister the service worker once (F12 → Application → Service Workers, or Clear site data) — PIIPS is a PWA and caches its static shell.

First-time server setup

Full detail lives in `Deploy_To_Network.txt` at the project root; summary:

1. **Python environment:** `python -m venv venv`, then `pip install --no-deps -r requirements.txt` (Python 3.10.x, 64-bit — other versions have no matching PaddleOCR/opencv wheels). `--no-deps` matters: requirements.txt is a full freeze, and re-resolving it fights the pinned `opencv-python==4.6.0.66` paddleocr needs.
2. **Frontend:** `cd frontend; npm install; npm run build` (skip if a pre-built `frontend/dist` was copied over).
3. **Windows service (NSSM):** install pointing at `venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000`, `AppDirectory` = the app folder, `Start SERVICE_AUTO_START`.
4. **IIS reverse proxy:** requires URL Rewrite + Application Request Routing, and ARR proxying enabled server-wide once (`appcmd set config -section:system.webServer/proxy /enabled:"True"` + `iisreset`) — without it the rewrite rule 404s instead of proxying. Open the site's port on the firewall, or it only loads from the server itself.

> **web.config.** If the server has the WebDAV IIS role feature installed, it intercepts POST/PUT/DELETE by default (GET still works) — every save action in the app fails with *Method Not Allowed* until it's removed. The shipped `web.config` already includes the fix (`<remove name="WebDAVModule"/>` under `<system.webServer> <modules>`); harmless when WebDAV isn't present.

Fresh database bootstrap (PIIPS_LIVE)

For a brand-new environment with no existing database, `PIIPS.sql` (built on the dev machine, not tracked in git — it contains a real password hash) drops-and-recreates the target database with the full current schema and master data only: user types, invoice types, statuses, a clean default Sadmin (`Sadmin@2026`), business templates, and a blank mail-settings password. Run it once against the target SQL Server, then:

1. Point Database Configuration at the new database and confirm the connection test succeeds.
2. Set Folder Configuration to the server's real working folder.
3. Open Mail Server Setting and enter the real SMTP password (the seeded row is deliberately blank — a DPAPI-encrypted password copied from another machine can't be decrypted here).
4. Sign in as Sadmin and change its default password, or create a personal Super Admin account.

Verification

```
Get-Service PIIPS_Backend                    # expect Running
Get-Website -Name "PIIPS_AI"                 # expect Started
Invoke-RestMethod http://127.0.0.1:8000/health
Invoke-WebRequest http://localhost:<port>/health -UseBasicParsing
Test-NetConnection <server-ip> -Port <port>   # from a DIFFERENT machine
```

Then in the browser: re-enter Database Configuration's connection string (a copied config.json can't be decrypted on a new machine), confirm Folder Configuration, enter the real Mail Server Setting password, and process one sample PDF end-to-end.

Common problems

SYMPTOM	FIX
502.3 right after a (re)start	PaddleOCR still initialising (10–30s) — wait and retry.
502.3 persistently	ARR proxy not enabled, or backend not listening on the port web.config targets.
404.4 (StaticFile, 0x8007007b) on the site's own port	Same root cause as above — ARR proxy isn't enabled.
Sign-in link in an email opens the wrong (internal) URL	Fixed this release — see the Developer Manual's Email chapter. Make sure the deployed app.py is current.
"Method Not Allowed" saving any form	WebDAV module intercepting non-GET verbs — see web.config note above.
Login fails: "db_connection is not configured"	config.json's connection string is empty or holds another machine's undecryptable DPAPI blob — re-enter it via Database Configuration (or set it in config.json directly, then restart the service; it re-encrypts automatically).
Site loads on the server but times out from elsewhere	No inbound firewall rule for the site's port.
"Data source name not found" (pyodbc)	ODBC Driver 17 for SQL Server isn't installed — the app hard-codes that exact driver name; a different version (e.g. 18) will not match.
First PDF fails, DNS error re: paddleocr.bj.bcebos.com	NSSM runs as Local System, whose model cache is separate from any interactive profile — let it download once, or pre-seed `...\systemprofile\.paddleocr`.
poppler/pdfinfo error	Poppler's `Library\bin` isn't on the system PATH.